R Quick Reference

STAT 109 — open-notes reference (Module 0–1 for Project 1; later modules below)

R Quick Reference

Module 0 — R basics

Task	Code pattern
Assign / print	`x <- 10` or `x = 10`, then `x` by itself. Note that `x` is a user chosen name.
Vector	`v <- c(1, 2, 3, 4)` a collection of objects: `c` is the R function, `v` is a user chosen name.
Index	`v[2]` (the element of `v` in the second position: R counts from 1)
Compare	`babies == mothers` returns TRUE/FALSE vector for each comparision of objects in the two sets of collections
Count matches	`sum(babies == mothers)` uses the trick that TRUE = 1 and FALSE = 0
One random shuffle	`sample(c(1, 2, 3, 4))`
Reproducible Randomness	`set.seed(109)`
Pre-allocate storage	`num_correct <- numeric(10000)`
Loop simulation	`for (i in 1:10000) { ... num_correct[i] <- ... }`
Estimate probability from the results of a for loop simulation	`mean(num_correct == 4)`
Counts of how many times each object appears	`table(num_correct)`

```r
# Module 0 - baby-matching simulation (pattern)
set.seed(109)
mothers <- c(1, 2, 3, 4)
num_correct <- numeric(10000)
for (i in 1:10000) {
  babies <- sample(c(1, 2, 3, 4))
  num_correct[i] <- sum(babies == mothers)
}
mean(num_correct == 4)   # P(all 4 correct)
mean(num_correct == 0)   # P(none correct)
```

Module 1 — Data structures & simulation

Task	Code pattern
Type check	`class(x)`
Integer	`y <- 3L`
Length	`length(v)`
Subset range	`v[2:4]`; drop 3rd: `v[-3]`

Task	Code pattern
Logical filter	`v[v > 5]`
Matrix	`m <- matrix(1:6, nrow = 2, ncol = 3)`
Matrix size	`dim(m)`
Matrix index	`m[2, 1]`; row 1: `m[1, ]`; col 2: `m[, 2]`
Membership	`rolls %in% c(2, 4, 6)`
Even die simulation	`mean(sample(1:6, 10000, replace = TRUE) %% 2 == 0)`

```r
v <- c(2, 4, 6, 8, 10)
v[3]
v[v > 5]
m <- matrix(1:6, nrow = 2, ncol = 3)
m[2, 1]
```

Module 1 — Conditional simulation

Task	Code pattern
Build labeled pool	`balls <- rep(c("R", "W"), c(3, 3))`
Draw without replacement	`sample(balls, size = 2, replace = FALSE)`
Pre-allocate	`successes <- vector("numeric", 10000)`
Record with `if`	`if (condition) { successes[k] <- 1 }`
AND / OR in `if`	`if (a & b)`; `if (a | b)`
Which replicates	`which(successes == 1)`
Weighted sample	`sample(c("yes", "no"), n, replace = TRUE, prob = c(0.7, 0.3))`
Joint probability	`mean(disease == 1 & test == 1)`
Conditional from sim	`mean(disease == 1 & test == 1) / mean(test == 1)`

```r
balls <- rep(c("R", "W"), c(3, 3))
replicates <- 10000
successes <- vector("numeric", replicates)
set.seed(109)
for (k in 1:replicates) {
  draw <- sample(balls, 2, replace = FALSE)
  if (draw[1] == "W" & draw[2] == "R") successes[k] <- 1
}
mean(successes)

# Joint event
mean(child == 1 & gummy == 1)
```

Module 1 — Binomial & Project 1

Task	Code pattern
Combinations	`choose(n, k)`
P(X = x)	`dbinom(x, size = n, prob = p)`
P(X q)	`pbinom(q, size = n, prob = p)`
P(X x) upper tail	`pbinom(x - 1, size = n, prob = p, lower.tail = FALSE)` or `1 - pbinom(x - 1, ...)`
Simulate binomial count	`rbinom(10000, size = n, prob = p)`
Load plotting	`library(tidyverse)`

Null binomial graph (milestone & final report)

```
library(tidyverse)

n <- 25          # your study
p <- 0.5         # from H0
x_obs <- 18      # observed successes after data collection

df <- data.frame(
  x = 0:n,
  prob = dbinom(0:n, size = n, prob = p)
)

ggplot(df, aes(x = x, y = prob)) +
  geom_col() +
  geom_vline(xintercept = x_obs, linewidth = 1) +
  labs(x = "x (successes)", y = "P(X = x) under H0")
```

p-value patterns (match H_a direction)

```
# Right-tailed Ha: p > p0  - at least x_obs successes
1 - pbinom(x_obs - 1, size = n, prob = p)

# Left-tailed Ha: p < p0
pbinom(x_obs, size = n, prob = p)

# Two-tailed (symmetric null) - tail at least as extreme as x_obs
# (use smaller of upper/lower doubled, or shade both tails on plot)
```

Descriptive chart for your data

```
# Bar chart of successes vs failures (example)
counts <- c(Success = x_obs, Failure = n - x_obs)
barplot(counts, main = "My sample", ylab = "Count")
```

Module 1 — Quick function index

Function	Role
`<-` or `=`	assign a value to a name (use `<-` in course code; `==` compares)
`c()`	combine into a vector
`class()`	data type
`length()`, `dim()`	size of vector / matrix
`matrix()`	2D numeric table
`sum()`, `mean()`	count TRUE; proportion TRUE
`sample()`	random draw; use `replace =`, `prob =`
`rep()`	repeat values to build a pool
`set.seed()`	reproducible randomness
`for`	repeat simulation
`if`	branch on condition
`which()`	indices where TRUE
`%in%`	element in set?
`&`, `|`	and / or (element-wise)
`choose()`	combinations
`dbinom()`, `pbinom()`, `rbinom()`	binomial pmf, cdf, simulate
`ggplot`, `aes`, `geom_col`, `geom_vline`	null distribution plot

Analysis pipeline

Step	Code idea	Quick check
1	`library(tidyverse)`	Packages load without errors
2	`read_csv(...); glimpse(dat)`	Column names and types match Part 1
3	`mutate(... factor(..., levels = ...))`	Levels match your two groups / outcome categories
4	`group_by` + `summarise` and/or `count`	Means/SDs if numerical; counts if categorical
5	`ggplot`	Boxplot/histogram for numerical by group; bar chart for categorical
6	**Test + CI**	`t.test(..., var.equal = FALSE)` or `prop.test` / `chisq.test`
7	Short interpretation	p-value addresses chance only

Setup

```
if (!requireNamespace("tidyverse", quietly = TRUE)) {
  install.packages("tidyverse", repos = "https://cloud.r-project.org")
}
library(tidyverse)
```

Import and inspect

Task	Code pattern
Load CSV	`dat <- read_csv("file.csv", show_col_types = FALSE)`
Peek structure	`glimpse(dat)`
First rows	`head(dat, 10)`

Categorical variables (factors)

Task	Code pattern
Set levels / order	`dat <- dat |> mutate(group = factor(group, levels = c("A", "B")))`
Check levels	`levels(dat$group)`

Summaries

Setting	Code pattern
One numerical variable	`dat |> summarize(mean_y = mean(y), sd_y = sd(y), n_y = n())`
Numerical **by two groups**	`dat |> group_by(group) |> summarize(n = n(), mean_y = mean(y), sd_y = sd(y))`
Counts for one categorical	`dat |> count(category) |> mutate(prop = n / sum(n))`
2 $\times$ **2** counts	`dat |> count(group, outcome)` then `pivot_wider`

Plots (ggplot2)

Setting	Pattern
Numerical response, **two groups**	`ggplot(dat, aes(x = group, y = y)) + geom_boxplot()`
Two categorical	`dat |> count(group, outcome) |> ggplot(aes(x = group, y = n, fill = outcome)) + geom_col()`
Two numerical	`ggplot(dat, aes(x = x1, y = x2)) + geom_point()`
Subclassification	Add `facet_wrap(~ confounder)`

Always add **`labs(title = ..., x = ..., y = ...)`** with **units** where relevant.

Two-sample tests and confidence intervals

Response type	R command	Notes
Numerical, 2 groups	`t.test(y ~ group, data = dat, var.equal = FALSE)`	Welch t-test; includes 95% CI
Binary, 2 groups	`prop.test(x = c(x1, x2), n = c(n1, n2), correct = FALSE)`	Compares two proportions
Two categorical variables	`chisq.test(tab)`	Check `chisq.test(tab)$expected` for 5 rule

Simulating small datasets

Task	Example pattern
Build tibble	`tibble(group = rep(c("A", "B"), each = 30), y = c(rnorm(30, 10, 2), rnorm(30, 12, 2)))`
Binary outcome	`rbinom(n, 1, prob = ...)` inside `mutate` or `c(...)`
Save	`write_csv(dat, "project4_data.csv")`
